service.yml file.

You can also try running the application with the `prod` profile enabled. Stop Catalog Service with Ctrl-C, and then start the application again with `prod` as an active profile:

```
$ java -jar build/libs/catalog-service-0.0.1-SNAPSHOT.jar \
    --spring.profiles.active=prod
```

The expected result is now the message defined in the catalog-service-prod.yml file in the `config-repo` repository:

```
$ http :9001/
Welcome to the production catalog from the config server
```

Once again, stop the previous application's execution with Ctrl-C.

The following section will cover how you can make the interaction between application and config server more fault-tolerant.

4.4.2 Making the configuration client resilient

When the integration with the config server is not optional, the application fails to start up if it cannot contact a config server. If the server is up and running, you could still experience issues due to the distributed nature of the interaction. Therefore it's a good idea to define some timeouts to make the application fail faster. You can use the `spring.cloud.config.request-connect-timeout` property to control the time limit for establishing a connection with the config server. The `spring.cloud.config.request-read-timeout` property lets you limit the time spent reading configuration data from the server.